

Vizsgaremek dokumentáció: Vidate társskereső és valós idejű kommunikációs rendszer

A jelen dokumentáció a Vidate elnevezésű társskereső és valós idejű kommunikációs rendszer teljes körű fejlesztői és felhasználói specifikációját, valamint tesztelési jegyzőkönyveit tartalmazza. A Vidate egy modern, skálázható architektúrára épülő, **élő videós WebRTC technológiát használó valós idejű videochat és videórandi alkalmazás**. A platform célja, hogy a felhasználók közötti interakciókat a hagyományos, szövegalapú társskeresőkkal ellentétben azonnali, személyes és dinamikus videós formátumban valósítsa meg. A rendszer a preferencialapú partnerkeresést és az azonnali vizuális kapcsolatfelvételt helyezi a középpontba, mentesítve a felhasználókat a hosszas várakozási időktől.

A projekt fejlesztése szigorúan szervezett munkamegosztásban zajlott. A fejlesztés során Zemen Dániel felelt a kliensoldali frontend felületek megtervezéséért és implementálásáért, a valós idejű folyamatok integrációjáért, valamint a Redis alapú üzenetközvetítő rendszer kialakításáért és a WebRTC-t használó videóhívások lebonyolításáért. Szabó Bernát feladatkörébe tartozott a szerveroldali backend architektúra megtervezése, az adatbázis sémáinak kialakítása és a komplex tesztelési folyamatok végrehajtása. Jelen dokumentációt közösen állítottuk össze.

Vizsgaremek dokumentáció: Vidate társkereső és valós idejű kommunikációs rendszer

I. Fejlesztői dokumentáció

I.1. Technológiai stack

I.2. Backend technológiák és infrastruktúra

I.3. Frontend technológiák és kliensoldali architektúra

I.4. Architektúra

I.5. ER Diagram és Adatmodellek

I.6. Class diagram és a repository minta

I.7. A matchmaking működése

I.8. REST API

I.9. Felhasználó bejelentkezése és Hitelesítés

I.10. Új felhasználói fiók regisztrációja

I.11. Társkeresési preferenciák módosítása

I.12. Beszélgetési előzmények lekérése lapozással

I.13. Új adminisztrátori fiók regisztrációja

I.14. Frontend komponensek bemutatása

I.15. Profilkezelő és onboarding modulok

I.16. Párosítási és Keresési Modulok

I.16. Valós Idejű Kommunikációs Komponensek

I.17. Élő Videó Randi (WebRTC) és Állapotkezelés

I.18. Manuális tesztek

Részletes tesztelési táblázat – Felhasználói felület (Frontend)

Részletes tesztelési táblázat – Adminisztrátori felület

I.19. Automatizált tesztek

II. Felhasználói dokumentáció

II.1. A probléma leírása

II.2. Szerepkörök bemutatása

II.3. Felhasználói esetek (Use Cases) és Üzleti Logika

Profil beállítása és preferencia logikák

Keresés indítása és a matchmaking algoritmus

Élő Videórandi (WebRTC) lebonyolítása

Moderáció és Biztonság

Biztonságos Fiók-helyreállítás és Jelszókezelés

III. Idézett munkák

I. Fejlesztői dokumentáció

A fejlesztői dokumentáció célja a szoftver belső működésének, alkalmazott technológiáinak és architektúráis döntéseinek mélyreható bemutatása. Ez a részletes leírás teszi lehetővé a rendszer későbbi karbantartását, a funkciók továbbfejlesztését, valamint az új fejlesztők számára a kódbázis megértését. A dokumentum követi a szoftverfejlesztési iparágban elfogadott szabványokat, kitérve a technológiai döntések okaira, az adatmodellek struktúrájára, az API végpontok specifikációjára, valamint a tesztelési eljárásokra.

I.1. Technológiai stack

A Vidate alkalmazás egy elosztott rétegeken alapuló, modern technológiai stacket alkalmaz, amely garantálja a magas rendelkezésre állást, a maximális típus biztonságot és a valós idejű adatfeldolgozást. A rendszer architektúrája egyértelműen elválasztja a kliensoldali megjelenítést a szerveroldali üzleti logikától és az adattárolástól, ezáltal biztosítva a komponensek független skálázhatóságát.

I.2. Backend technológiák és infrastruktúra

A szerver oldali alkalmazás alapja a **FastAPI** keretrendszer, amely egy modern, kiemelkedően gyors és aszinkron **Python** alapú webes keretrendszer.² A FastAPI választásának elsődleges oka a natív aszinkron programozási modell teljes körű támogatása. Egy valós idejű alkalmazás esetében, ahol nagyszámú nyitott **WebSocket** kapcsolatot kell egyidejűleg fenntartani, az **aszinkron** event loop használata elengedhetetlen a megfelelő teljesítmény eléréséhez. A FastAPI a Starlette mikroszolgáltatási keretrendszerre épül, amely aszinkron bemeneti és kimeneti (I/O) műveleteket tesz lehetővé, így a szerver nem blokkolódik a hálózati kommunikáció vagy az adatbázis lekérdezések során. A keretrendszer beépített **Pydantic** integrációja révén automatikus adatvalidációt biztosít, jelentősen csökkentve a hibás bemeneti adatokból eredő szerveroldali kivételeket. Továbbá a FastAPI automatikusan generálja a Swagger UI és ReDoc alapú interaktív API dokumentációkat az OpenAPI szabvány szerint, amely megkönnyíti a kliensoldali integrációs munkát és az API végpontok tesztelését.

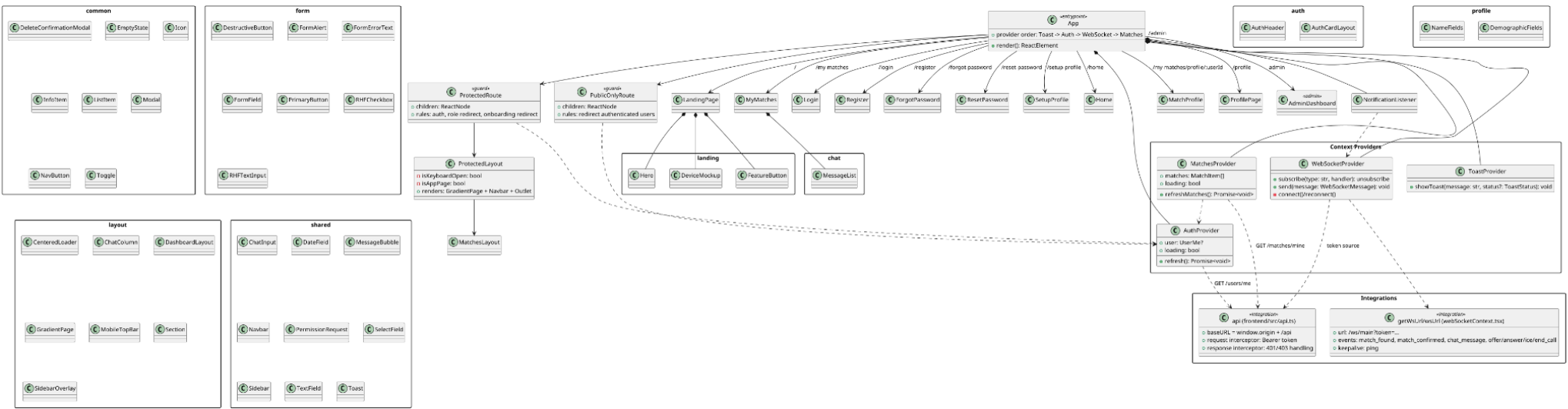
Az adatbázis kezelés és az objektum relációs leképezés a **SQLModel** könyvtárra épül. A SQLModel egy innovatív technológia, amely a **SQLAlchemy** robusztus adatbázis műveleteit és a Pydantic adatvalidációs képességeit ötvözi. Ez a megközelítés megszünteti az adatszerkezetek kód duplikációt, mivel ugyanazok az osztályok

használhatók mind az adatbázis tábláinak sémájaként, mind pedig a hálózati rétegen közlekedő JSON adatok validálására. A SQLAlchemy használatával a fejlesztési folyamat során a típusbiztonság a teljes backend logikán keresztül megmarad, megelőzve az adattípusok eltéréséből adódó rejtett hibákat.

Az adattárolásért egy **PostgreSQL** relációs adatbázis kezelő rendszer felel. A PostgreSQL kiválasztását a magas megbízhatóság, az ACID (atomicitás, konzisztencia, izoláció, tartósság) tranzakciók szigorú betartása és a komplex lekérdezések gyors végrehajtása indokolta. A Vidate alkalmazásban a partner keresési algoritmusok nagyszámú több a többhöz (many to many) kapcsolatot használnak a preferenciák szűrésére, amelyeket a PostgreSQL optimalizált lekérdezés tervezője rendkívül hatékonyan képes kiszolgálni.

A valós idejű kommunikáció, a matchmaking és átmeneti adatok (GPS koordináták, jelszó visszaállítási tokenek) tárolásának valamint a WebSocket szerver példányok közötti üzenetközvetítés motorja a **Redis**. A rendszer horizontális skálázhatóságának biztosítása érdekében a memóriában tároló Redis Pub/Sub architektúráját integráltuk a rendszerbe. Amikor egy felhasználó üzenetet küld a WebSocket kapcsolaton keresztül, a fogadó FastAPI szerver közzéteszi (publish) az üzenetet egy specifikus Redis csatornán. A többi szerver példány fel van iratkozva (subscribe) erre a csatornára, így amint megkapják az üzenetet, azonnal továbbítják azt a náluk csatlakozott cél felhasználónak. Ez a megoldás kiküszöböli a szerverek közötti közvetlen kommunikáció szükségességét, és lehetővé teszi, hogy az üzenetek szinte azonnal, minimális késleltetéssel jelenjenek meg a képernyőn. A Redis továbbá szerepet játszik a felhasználók online jelenlétének nyomon követésében is a matchmaking kivitelezéséhez.

A projekt telepítési és futtatási környezeteinek konzisztenciáját **Docker** technológia biztosítja. A Docker Compose segítségével az adatbázis, a Redis szerver, a FastAPI backend és az Nginx load balancer és reverse proxy különálló, izolált konténerekben futnak. Ez garantálja, hogy az alkalmazás bármely fejlesztői vagy éles környezetben megegyezően viselkedjen, kiküszöbölve a környezeti konfigurációkból adódó anomáliákat. A biztonságos jelszókezelésért az Argon2id algoritmus felel, amely titkosítja a felhasználói jelszavakat, kivédve ezáltal a nyers erőn alapuló (brute force) támadásokat. A frontend statikus fájljait az Nginx segítségével juttatjuk el a felhasználókhoz.



Frontend komponents diagram

I.3. Frontend technológiák és kliensoldali architektúra

A kliensoldali alkalmazás a népszerű **React** könyvtárra épül, amely deklaratív, komponensalapú architektúrájával lehetővé teszi a felhasználói felület kis, újrahasznosítható, független építő elemekre történő felbontását. A modern React alkalmazásokhoz híven az alkalmazás a funkcionális komponensekre és a React Hookokra támaszkodik, mellőzve a régebbi osztály alapú megoldásokat. A fejlesztési folyamat a Vite keretrendszerben történik, amely a hagyományos modulcsomagolóknál jóval gyorsabb hidegindítást és azonnali hot module replacement funkciót biztosít, jelentősen felgyorsítva a felhasználói felület finomhangolását.

A teljes frontend kód **TypeScript** nyelven íródott, amely kiterjeszti a JavaScriptet opcionális statikus típus megadással. A TypeScript alkalmazása elengedhetetlen egy olyan komplex, adatvezérelt alkalmazásnál, mint a Vidate. A statikus típus ellenőrzés a fordítási időben képes azonosítani a lehetséges hibákat, különösen a backendről érkező, több egymásba ágyazott objektumot tartalmazó JSON adatszerkezetek feldolgozásakor. A rendszer domain modelljei szigorúan követik a backend SQLModel sémáit, így garantálva az adatok típus biztonságát a hálózati kommunikáció mindkét végén. A TypeScript interfészek és típus aliasok használatával a kód olvashatósága és karbantarthatósága nagyságrendekkel javult.

A felhasználói felület modern, letisztult és reszponzív megjelenítéséért a **Tailwind CSS** és a **Flowbite** React komponenskönyvtár felelős. A Tailwind CSS egy utility first megközelítésű keretrendszer, amely lehetővé teszi a stílusok közvetlen alkalmazását a HTML elemeken dedikált osztálynevek segítségével, anélkül, hogy különálló, nehezen karbantartható stíluslapokat kellene írni. Ez a módszer rendkívül gyors prototípus készítést és egységes dizájn nyelvet tesz lehetővé. A Flowbite React a Tailwind CSS alapjaira építve kínál előre gyártott, hozzáférhetőségi (accessibility) szabványoknak megfelelő komponenseket, mint például modális ablakok, lenyíló menük, figyelmeztető kártyák és űrlap elemek.

A kliensoldali adatvalidáció kritikus fontosságú a szerver terhelésének csökkentése és a gyors felhasználói visszajelzések érdekében. Ezt a feladatot a **Zod** séma validációs könyvtár látja el, amely zökkenőmentesen integrálódik a TypeScript típus rendszerébe és a React Hook Form könyvtárba. A Zod segítségével az űrlapok beviteli mezői azonnali hibaüzeneteket generálnak, ha a felhasználó például érvénytelen e-mail formátumot ad meg, vagy ha a megadott életkori korlátok logikailag helytelenek, mindezt még a hálózati kérés elindítása előtt.

Az alkalmazás kliensoldali útválasztásáért (routing) a **React Router DOM** felel. Ez a technológia biztosítja, hogy a weboldal egyetlen oldalként (Single Page Application) működjön, elkerülve a teljes oldal frissítéseket navigáció közben. A React Router segítségével valósítottuk meg a komplex útvonalvédelmi (guard) rendszert is, amely garantálja, hogy a hitelesítést igénylő oldalakat csak a bejelentkezett felhasználók érhessék el, míg a bejelentkezési felületet ne láthassák azok, akik már aktív munkamenettel rendelkeznek.

Az alkalmazás videó feed oldalán található élő hívásokat a WebRTC technológia segítségével oldottuk meg, amely az élő videós kapcsolatok, a videochat és a videórandik valós idejű lebonyolítását teszi lehetővé. Ez a technológia biztosítja, hogy a videó- és hangadatok a lehető legkisebb késleltetéssel, közvetlenül a felhasználók böngészői között (peer-to-peer) áramoljanak.

A videókapcsolat felépítésének pontos lépései a következők:

1. **Jelzés (Signaling):** A backend WebSocket kapcsolatain keresztül történik a hálózati metaadatok (SDP - Session Description Protocol ajánlatok és válaszok, valamint ICE candidate-ek) cseréje.
2. **Médiafolyam (Media Stream) megnyitása:** A kliensoldali szoftver a böngésző natív API-jain keresztül engedélyt kér a felhasználó kamerájához és mikrofonjához. A jóváhagyást követően a rendszer a WebRTC API segítségével digitalizálja, tömöríti és optimalizálja a valós idejű médiafolyamot a hálózati átvitelhez.
3. **Közvetlen kapcsolatfelépítés (NAT Traversal):** Mivel a felhasználók túlnyomó többsége NAT (Network Address Translation) tűzfalak mögött, privát IP-címeken helyezkedik el, a közvetlen kapcsolat kialakításához az ICE (Interactive Connectivity Establishment) keretrendszert alkalmazzuk. A hálózati végpontok feltérképezését és az akadályokon való átjutást STUN, illetve szükség esetén TURN szerverek segítik. Ez a mechanizmus garantálja, hogy a videó- és hangadatok minden hálózati körülmény között a leoptimálisabb és legrövidebb úton jussanak el a partnerhez, biztosítva a kiváló minőségű, akadásmentes randiélményt.

Illetve a böngésző opcionálisan engedélyt kér a felhasználótól a valós idejű helyadatok használatához is. A koordinátákat a Redis-ben tároljuk el, mindaddig amíg a user abba nem hagyja a videó feed használatát. Ez segíti a matchmaking még személyre szabottabb működését.

I.4. Architektúra

A rendszer architektúrája egy klasszikus háromrétegű modellt követ, amely tiszta elkülönítést biztosít a prezentációs réteg, az üzleti logika és az adatelérési réteg között. A prezentációs réteg (API végpontok) kizárólag a hálózati kérések fogadásáért, a bemeneti adatok validálásáért és a válaszok formázásáért felel. Az üzleti logikai réteg (Service réteg és háttérfolyamatok) tartalmazza a partnerkeresési algoritmusokat, a jelszótitkosítást és a jogosultságkezelést. Az adatelérési réteg (Repository minta) pedig kizárólag az adatbázissal való kommunikációt absztrahálja el, így az üzleti logika teljesen független marad a konkrét adatbázis technológiától.

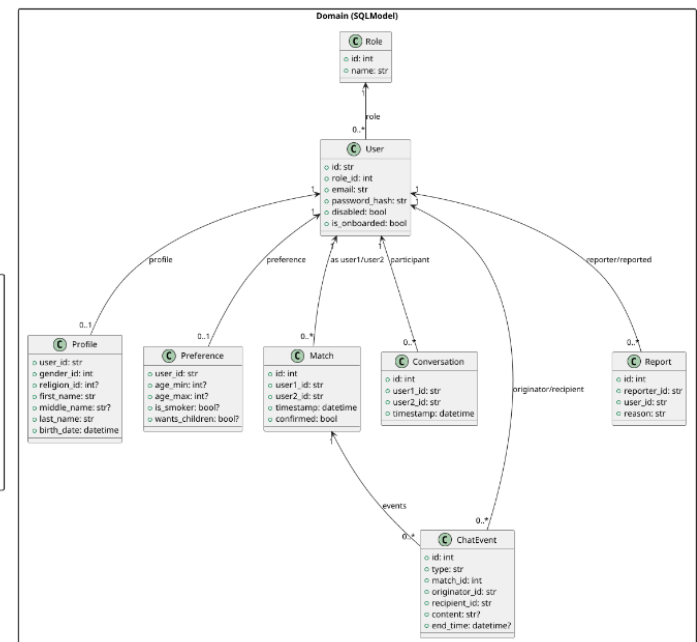
I.5. ER Diagram és Adatmodellek

A PostgreSQL relációs adatbázis sémája úgy lett megtervezve, hogy támogassa a nagy teljesítményű lekérdezéseket, a komplex szűrési feltételeket, valamint a szigorú adatintegritást. A relációs modell normalizált (3NF) formában épül fel, minimalizálva az adat redundanciát. Az adatbázis főbb entitásai és azok kapcsolatai a következők:

1. A **Users** tábla a rendszer legfőbb központi entitása. Itt tárolódnak a felhasználók alapvető hitelesítési adatai, a titkosított jelszó hash értékek, az e-mail címek és a fiók státuszára vonatkozó metaadatok. Ez a tábla áll a legtöbb kapcsolat középpontjában, és az idegen kulcsok hivatkozásainak elsődleges célpontja.
2. A **Preferences** tábla képezi a társkeresési algoritmus legfontosabb adathalmazát. Egy az egyhez kapcsolatban áll a Users táblával, oly módon, hogy a user_id mező egyszerre funkcionál elsődleges kulcsként és a Users táblára mutató idegen kulcsként. Ez az architekturális döntés biztosítja, hogy egy felhasználónak garantáltan csak egyetlen aktív preferencia készlete lehessen. A tábla tartalmazza a partnerkeresési korlátokat: az elvárt minimum és maximum életkort, a gyermekvállalási hajlandóságot, valamint a dohányzási szokásokra vonatkozó elvárásokat. Ezek a mezők logikai típusúak vagy egész számok, amelyek gyors összehasonlító lekérdezéseket tesznek lehetővé a háttérfolyamatok számára.
3. A rendszer több a többhöz kapcsolatokat használ a komplex demográfiai preferenciák kezelésére. A statikus opciókat **szótár táblák** (Lookup Tables) tárolják, ilyenek a **genders, languages és religions** táblák. Ezekhez a szótártáblákhoz a Preferences entitás kapcsolótáblákon keresztül (Association Tables) kötődik. A **PreferenceGenderLink, PreferenceLanguageLink és PreferenceReligionLink** táblák két idegen kulcsot tartalmaznak, amelyek együttesen alkotnak egy összetett elsődleges kulcsot, megelőzve az ismétlődő rekordok beszúrását. Amikor a rendszer egy felhasználó preferenciáit kéri le, az ORM réteg automatikusan végrehajtja a szükséges illesztéseket (SQL JOIN), és egy strukturált objektum gráffá fűzi össze az adatokat.
4. A **Matches** tábla a két felhasználó közötti sikeres párosítást reprezentálja. Ez a tábla kapcsolótáblaként funkcionál a Users tábla felett önmagára hivatkozva, mivel két különböző felhasználót köt össze. Tartalmazza a user1_id és user2_id mezőket, egy kötelező UTC időbélyeget a létrehozás idejével, valamint egy confirmed logikai mezőt, amely a párosítás kölcsönös elfogadását hivatott jelezni. A relációs adatintegritás megőrzése érdekében az adatbázisban egy összetett egyedi kényszer (UniqueConstraint) van definiálva a két felhasználói

azonosító kombinációjára, amely garantálja, hogy két felhasználó között csak egyetlen aktív párosítási rekord jöhessen létre.

5. A kommunikációs adatok perzisztens tárolásáért a **Conversations** és a **ChatEvents** táblák felelnek. A Conversations tábla rögzíti magát a kommunikációs szálát a két érintett felhasználó azonosítójával és az utolsó aktivitás időbélyegével. A tényleges üzeneteket a ChatEvents tábla tárolja, amely külső kulccsal hivatkozik a beszélgetésre és a küldő felhasználóra.



I.6. Class diagram és a repository minta

Az adatbázis műveletek elvont, objektumorientált kezelésére a projekt a Repository tervezési mintát alkalmazza. Ez a megközelítés lehetővé teszi, hogy a FastAPI útvonalválasztói (router-ei) és a szolgáltatási réteg ne tartalmazzanak közvetlen SQL lekérdezéseket vagy ORM specifikus kódokat, ezzel javítva a kód tesztelhetőségét és olvashatóságát.

A struktúra alapját a **BaseRepo** generikus őssztály (Pythonban Generics típusú osztály) adja. Ez az osztály implementálja az összes alapvető adatbázis műveletet: entitások létrehozását, egyedi azonosító alapján történő lekérdezését, frissítését és törlését. Minden specifikus repozitórium ebből az őssztályból származik le, megörökölve a típusbiztos metódusokat és a központi adatbázis munkamenet kezelési logikáját.

A **ChatEventRepo** osztály a BaseRepo leszármazottja, amely kiegészül a csevegőalkalmazásokra jellemző komplex, **lapozást** végző lekérdezésekkel. Ennek az osztálynak a legkritikusabb aszinkron metódusa a görgetés alapú lapozást megvalósító lekérdezés. A hagyományos offset alapú lapozás helyett, amely nagy adathalmazoknál lassulást okozhat, a metódus egy egyedi azonosítót kap bemenetként. A lekérdezés megvizsgálja a relációs adatbázisban a megadott azonosítót, majd annál szigorúan kisebb azonosítójú rekordokat keres. A találatokat ezután csökkenő sorrendbe rendezi, és a megadott korlátig levágja az eredményhalmazt. Ez a mechanizmus garantálja az állandó válaszidőt függetlenül attól, hogy a beszélgetés hány ezer üzenetet tartalmaz.

A **MatchRepo**, **GenderRepo** és **LanguageRepo** osztályok szintén a BaseRepo implementációi. Míg a szótártáblákat kezelő repozitóriumok elsősorban olvasási műveleteket végeznek, a MatchRepo összetett szűrési logikákat is tartalmaz. Ide tartozik a megerősítetlen párosítások lekérdezése, vagy a felhasználó összes aktív kapcsolatának visszaadása egyetlen, optimalizált SQL lekérdezés formájában.

I.7. A matchmaking működése

A rendszer háttérben futó magja a **Matchmaking** osztály, amely az üzleti logikai rétegben (Business Logic Layer) helyezkedik el. Ez a komponens felelős a felhasználók összekötéséért az adatbázisban tárolt szabályok alapján. A hagyományos, centralizált "cron-job" jellegű megközelítés helyett ez az osztály aszinkron feladatként (task) funkcionál: minden egyes keresést indító felhasználóhoz egy önálló, alacsony

erőforrásigényű folyamatot rendel, amely másodpercenkénti időközönként fut le a szerveren. Ez a decentralizált modell biztosítja, hogy a keresési ciklus azonnal elinduljon a csatlakozáskor, felesleges várakozási idők nélkül.

Amikor a Matchmaking folyamat lefut, egy összetett memóriabeli algoritmus segítségével választja ki az ideális partnert. A folyamat lépései és az osztály belső logikája a következőképpen épül fel:

1. **Adatok lekérése és hibrid gyorsítótárazás:** Az osztály először lekérdezi a felhasználó aktív **Preferences** objektumát és kapcsolótábláit a PostgreSQL adatbázisból. Ezzel párhuzamosan az éppen elérhető, párosításra váró partnerek (matchmaking pool) listáját és azok geolokációs adatait a memóriában futó Redis-ből (illetve a RediSearch modulból) nyeri ki. Ez a megközelítés tehermentesíti a relációs adatbázist a másodpercenkénti többbezzres lekérdezéstől.
2. **Kizáró feltételek (Hard Constraints) érvényesítése:** Az algoritmus első lépésben a RediSearch modul segítségével kiszűri azokat a jelölteket, akikkel a párosítás fizikailag vagy szabályzatilag lehetetlen. Ide tartozik a nemek szerinti kötelező egyezés ellenőrzése, valamint a moderációs rendszer által korábban letiltott (blocked) felhasználók azonnali kizárása. Fontos, hogy először mindig a 100 legrégebb óta várakozó jelölt között keres az algoritmus.
3. **Többtényezős pontozás (Soft Constraints):** A bináris szűrés után megmaradt jelölteket az osztály egy súlyozott pontrendszer alapján értékeli. A cél a "nulla találat" állapot elkerülése, így az algoritmus rugalmasan kezeli a preferenciákat:
 - *Interakciós büntetés:* Az osztály megvizsgálja a korábbi beszélgetések naplóját. Hatalmas pontlevonással sújtja azokat a jelölteket, akikkel a felhasználó a közelmúltban már beszélt, így megakadályozva a végtelenített, ismétlődő párosításokat.
 - *Demográfiai és nyelvi bónuszok:* A beállított korosztályba eső jelöltek, illetve a közös nyelvet beszélő partnerek magas bónusz pontokat kapnak.
 - *Geolokáció és életmód:* A Redis geolokáció funkciói alapján kiszámított fizikai távolság, valamint a kiegészítő értékek (vallás, dohányzás stb.) tovább finomítják az eredményt.
4. **Atomi tranzakció és versengés kezelés (Race Condition Protection):** Miután az algoritmus kiválasztotta a legmagasabb pontszámot elérő, optimális jelöltet, a **Matchmaking** osztály egy beágyazott Lua scriptet futtat le a Redis szerveren. Mivel a rendszerben mindenki egyszerre aktív kereső és választható jelölt, fennáll a veszélye, hogy két különböző háttérfolyamat ugyanazt az embert próbálja lefoglalni. A Lua script garantálja, hogy a jelölt kiválasztása, ellenőrzése és a poolból való eltávolítása egyetlen, megszakíthatatlan (atomi) műveletként

menjen végbe. Ha egy másik fél folyamata gyorsabb volt, a script visszautasítja a kérést, és az algoritmus a következő legjobb jelölttel próbálkozik.

5. **A találat véglegesítése:** Amennyiben az atomi foglalás sikeres és az elvárások kölcsönösen teljesültek, az osztály tranzakcionális biztonsággal létrehoz egy új **Match** entitást az SQL adatbázisban. Végezetül értesítést küld a valós idejű kommunikációs modul felé: a Redis **PUBLISH** parancsán keresztül szignált küld a WebSocket szervereknek, amelyek azonnal eljuttatják a sikeres párosításról szóló adatokat mindkét érintett felhasználó képernyőjére
6. **WebRTC signaling és kapcsolatfelépítés:** A sikeres párosításról szóló WebSocket értesítés megérkezését követően a kliensoldalon automatikusan elindul a WebRTC signaling (jelzési) folyamat. A két résztvevő kliens a rendszeren keresztül a háttérben kicseréli a közvetlen csatlakozáshoz elengedhetetlen hálózati metaadatokat (SDP – Session Description Protocol ajánlatok és válaszok, valamint az ICE candidate-ek). Ennek a folyamatnak a lezárásaként sikeresen feláll a közvetlen, peer-to-peer WebRTC videó- és hangkapcsolat a két felhasználó között.

I.8. REST API

Az alkalmazás backend rendszere egy teljes értékű, **aszinkron REST API**-t biztosít a kliens alkalmazások számára. Az összes végpont szigorú **JSON** alapú kommunikációt használ, és a bemeneti adatok validálása a Pydantic modelleken keresztül történik. A végpontok védelmét JSON Web Token technológia biztosítja, amely minden privát végpont esetén az Authorization HTTP fejlécben érkezik. A következőkben a rendszer legfontosabb végpontjainak részletes működése, üzleti logikája és adatszerkezete kerül bemutatásra.

(Az összes endpoint OpenAPI (swagger) dokumentuma megtalálható [yaml formátumban](#) és [képként](#) a githubon.)

I.9. Felhasználó bejelentkezése és Hitelesítés

A bejelentkezési végpont felelős a felhasználók hitelesítéséért és a hozzáférési tokenek kiállításáért. A biztonság kritikus szempont, ezért a rendszer szigorú ellenőrzési folyamatokon vezet keresztül a beérkező adatokat.

- **Útvonal és HTTP Metódus:** POST /api/auth/login
- **A folyamat részletes narratívája:** Amikor a kliens alkalmazásból a felhasználó

elküldi a hitelesítési adatait, a FastAPI router azonnal átadja a kérést az autentikációs szolgáltatásnak. A szolgáltatás első lépésben validálja, hogy a kérés tartalmazza-e a szükséges mezőket. Ezt követően lekérdezi az adatbázisból a megadott e-mail címhez tartozó felhasználói rekordot. Ha a felhasználó nem létezik, a rendszer biztonsági okokból egy generikus hibaüzenetet ad vissza, hogy megelőzze a fiókok létezésének feltérképezését. Amennyiben a rekord létezik, a backend alkalmazza az Argon2id kriptográfiai algoritmust a beérkező jelszón. Az így kapott eredményt összehasonlítja az adatbázisban tárolt lenyomattal (hash-sel). Egyezés esetén a szolgáltatás létrehoz egy JWT tokenet, amely tartalmazza a felhasználó egyedi azonosítóját és a token lejáratí idejét. Ezt a tokenet digitálisan aláírja a szerver titkos kulcsával, majd a választ JSON formátumban visszaküldi a kliensnek.

- **Hibakezelés:** Ha a jelszó vagy az email cím nem egyezik, a rendszer 401 Unauthorized státuszkóddal tér vissza. Formátumhibás kérés esetén a Pydantic automatikusan 422 Unprocessable Entity hibát generál.

Kérés Törzse (Request Body)	Adattípus	Leírás	Kötelező
email	String	A felhasználó regisztrált e-mail címe	Igen
password	String	A felhasználó titkos jelszava	Igen

Válasz Törzse (Response - 200 OK)	Adattípus	Leírás
access_token	String	A generált és aláírt JWT hozzáférési token
token_type	String	A token típusa, jellemzően "bearer"
role	String	A felhasználóhoz rendelt rendszerbeli szerepkör

I.10. Új felhasználói fiók regisztrációja

Ez a végpont biztosítja az új látogatók csatlakozását a platformra. Az adatok konzisztenciájának és a rendszer biztonságának fenntartása érdekében a regisztrációs folyamat komplex validációt hajt végre.

- **Útvonal és HTTP Metódus:** POST /api/auth/register
- **A folyamat részletes narratívája:** A regisztrációs űrlap beküldésekor a backend elsődleges feladata az adatok formátumának ellenőrzése. A FastAPI végpont automatikusan futtatja a Pydantic modell validációs logikáját, amely reguláris kifejezésekkel ellenőrzi az e-mail cím formátumát, és megvizsgálja, hogy a jelszó és a megerősítő jelszó pontosan megegyezik-e. Ha a validáció sikeres, a rendszer egy gyors adatbázis-lekérdezést indít, hogy ellenőrizze, létezik-e már felhasználó az adott e-mail címmel. Ez a lépés elengedhetetlen a fiók duplikációk elkerülése végett. Ütközés esetén a folyamat megszakad. Ha az e-mail cím szabad, a szerver legenerálja a jelszó kriptográfiai lenyomatát (hash-t), majd egy atomi adatbázis tranzakció keretében beszúrja az új rekordot a Users táblába. A sikeres beszúrás követően a rendszer egy visszaigazoló JSON objektumot küld, amely tartalmazza a sikeres végrehajtást igazoló üzenetet.
- **Hibakezelés:** Amennyiben a megadott e-mail cím már szerepel az adatbázisban, a rendszer 400 Bad Request státuszkóddal és "This email is already registered." üzenettel tér vissza. Formátumhibák esetén 422 Unprocessable Entity érkezik.

Kérés Törzse (Request Body)	Adattípus	Leírás	Kötelező
email	String	Érvényes formátumú, nem foglalt e-mail cím	Igen
password	String	A biztonsági követelményeknek megfelelő jelszó	Igen
confirmPassword	String	A jelszó pontos másolata ellenőrzés céljából	Igen

Válasz Törzse (Response - 201 Created)	Adattípus	Leírás
message	String	A művelet sikerességét igazoló szöveges üzenet
redirect	String	A frontend számára javasolt átirányítási útvonal

I.11. Társkeresési preferenciák módosítása

Ez a végpont felelős a felhasználó partnerkeresési elvárásainak frissítéséért. Mivel ez a folyamat több adatbázis táblát is érint egyidejűleg, a tranzakció biztonság fenntartása kiemelten fontos.

- **Útvonal és HTTP Metódus:** PATCH /api/preferences
- **A folyamat részletes narratívája:** A végpont eléréséhez érvényes hitelesítési token szükséges. A szerver a kérés fejlécéből kinyeri a JWT tokenet, dekódolja azt, és azonosítja a módosítást kezdeményező felhasználót. Ezt követően a rendszer kiértékeli a beküldött preferenciákat. Egy beágyazott üzleti logikai ellenőrzés fut le, amely megvizsgálja, hogy a megadott minimum életkor logikailag nem haladja-e meg a maximum életkort. Sikeres ellenőrzés után az ORM (SQLModel) egyetlen tranzakciót indít az adatbázis felé. Először frissíti a fő Preferences táblában az alapvető értékeket (életkorok, dohányzási szokások, gyermekvállalási szándék). A tömbként érkező adatok (nemek, nyelvek, vallások) esetén a szerver először törli az adott felhasználóhoz tartozó korábbi bejegyzéseket a megfelelő kapcsolótáblákból (PreferenceGenderLink, stb.), majd tömeges beszúrással (bulk insert) hozzáadja az új azonosítókat. A tranzakció sikeres lezárása után az API a frissített, teljes preferencia objektumot adja vissza.
- **Hibakezelés:** Érvénytelen token esetén 401 Unauthorized hibakódot kap a kliens. Ha az age_min nagyobb, mint az age_max, a rendszer 400 Bad Request kóddal és egy specifikus, felhasználóbarát hibaüzenettel tér vissza.

Kérés Törzse (Request Body)	Adattípus	Leírás	Kötelező
age_min	Integer	A potenciális partnertől elvárt minimum életkor	Nem
age_max	Integer	A potenciális partnertől elvárt maximum életkor	Nem
wants_children	Boolean	Elvárás a gyermekvállalási hajlandóságra	Nem
is_smoker	Boolean	Elvárás a partner dohányzási szokásaira	Nem
gender_ids	Array[Integer]	A preferált nemek azonosítóinak listája	Nem
language_ids	Array[Integer]	A preferált beszélt nyelvek listája	Nem

Válasz Törzse (Response - 200 OK)	Adattípus	Leírás
preferences	Object	A teljes, sikeresen mentett preferencia adatszerkezet

I.12. Beszélgetési előzmények lekérése lapozással

Ez a végpont biztosítja a felhasználók közötti kommunikáció folyamatosságát azáltal, hogy optimalizált, lapozható formában szolgáltatja a korábbi üzeneteket.

- **Útvonal és HTTP Metódus:** GET /api/chat/messages/{match_id}
- **A folyamat részletes narratívája:** A kliens a kérés URL-jében továbbítja annak a kapcsolatnak az azonosítóját (match_id), amelynek a beszélgetését meg kívánja tekinteni. A biztonsági réteg első feladata annak ellenőrzése, hogy a lekérdezést indító, JWT token alapján azonosított felhasználó valóban résztvevője-e a kért párosításnak. Jogosulatlan próbálkozás esetén a kérés azonnal elutasításra kerül. Ha a jogosultság rendben van, a rendszer feldolgozza a query paraméterként érkező last_id és limit értékeket. A ChatEventRepo osztály lekérdezést indít az adatbázis felé: kizárólag a megadott last_id értéknél régebbi (kisebb azonosítójú) üzeneteket szűri le a ChatEvents táblából, és azokat a legutóbbiaktól visszafelé rendezve (DESC szerinti rendezés) adja vissza, figyelembe véve a megadott korlátot (limit). Végül a szerver visszaadja a kinyert üzenetek tömbjét, valamint egy logikai jelzőt, amely tájékoztatja a klienst, hogy vannak-e még további betölthető üzenetek a szerveren.
- **Hibakezelés:** Ha a kért match_id nem létezik, a szerver 404 Not Found hibát jelez. Ha a felhasználó nem résztvevője a beszélgetésnek, 403 Forbidden státuszkóddal tagadja meg a hozzáférést.

URL Paraméterek (Path & Query)	Adattípus	Leírás	Kötelező
match_id	Integer	A beszélgetéshez tartozó párosítás azonosítója (Path)	Igen
last_id	Integer	Az eddig betöltött legrégebbi üzenet azonosítója (Query)	Nem
limit	Integer	A lekérdezni kívánt üzenetek maximális száma (Query)	Nem

Válasz Törzse (Response - 200 OK)	Adattípus	Leírás
messages	Array[Object]	Az üzenet objektumokat tartalmazó lista
has_more	Boolean	Igaz, ha maradt még betölthető régebbi üzenet

I.13. Új adminisztrátori fiók regisztrációja

Ez a végpont biztosítja az új adminok csatlakozását a platformra. Az admin regisztrációs linkel való visszaélés elkerülése érdekében egy egyszer használatos (one-time) tokennel lehet igénybe venni, melyet a linknek kell tartalmaznia.

- **Útvonal és HTTP Metódus:** POST /api/auth/register-admin
- **A folyamat részletes narratívája:** A regisztrációs űrlap beküldésekor a backend ellenőrzi hogy létezik-e már felhasználó ilyen email címmel. Ha az e-mail cím szabad, a szerver ellenőrzi a token érvényességét (azzal, hogy megnézi, tartalmazza-e a Redis ezt a tokent) ezzel biztosítva, hogy azt csak egyszer lehessen felhasználni. Ezt követően a rendszer dekódolja a JWT tokent, és ellenőrzi annak típusát (kizárólag a register-admin típus fogadható el). Sikeres validáció esetén a szerver legenerálja a jelszó kriptográfiai lenyomatát (hash-t), a felhasználóhoz rendeli az admin szerepkört (role), majd beszúrja az új rekordot a megfelelő adatbázis táblába. A sikeres beszúrást követően a rendszer log-olja az eseményt és a folyamatot lezárja.
- **Hibakezelés:** Amennyiben a megadott e-mail cím már szerepel az adatbázisban, a rendszer 400 Bad Request státuszkóddal és "This email is already registered." üzenettel tér vissza. Ha a token már felhasználásra került (vagy érvénytelen), 401 Unauthorized és "Token already used" üzenet érkezik. Nem megfelelő token típus esetén szintén 401 Unauthorized érkezik "Wrong token" üzenettel. Formátumhibák esetén 422 Unprocessable Entity kerül kiküldésre.

URL Paraméterek (Path & Query)	Adattípus	Leírás	Kötelező
token (Query)	String	Egyszer használatos, JWT formátumú admin regisztrációs kulcs	Igen
email (Body)	String	Érvényes formátumú, nem foglalt e-mail cím	Igen
password (Body)	String	A biztonsági követelményeknek megfelelő jelszó	Igen
confirmPassword (Body)	String	A jelszó pontos másolata ellenőrzés céljából	Igen

Válasz Törzse (Response - 200 OK)	Adattípus	Leírás
message	String	A művelet sikerességét igazoló szöveges üzenet
redirect	String	A frontend számára javasolt átirányítási útvonal

I.14. Frontend komponensek bemutatása

A Vidate kliens alkalmazás felhasználói felülete egy precízen megtervezett, moduláris React architektúrára épül. A komponensek szigorúan funkcionális alapon vannak csoportosítva, amely lehetővé teszi a független fejlesztést és a kód újrahasznosíthatóságát. A TypeScript alkalmazása minden komponens esetében biztosítja, hogy a rajtuk átfolyó adatok formátuma (prop-ok) fordítási időben validált legyen.

I.15. Profilkezelő és onboarding modulok

A rendszerbe történő belépést követően a legfontosabb felület a felhasználói adatok bekérése és megjelenítése. Ez a folyamat több kisebb, dedikált komponensből épül fel. A **setupProfile** komponens egy komplex űrlapoldal, amely fogadja az újonnan regisztrált felhasználókat. Feladata, hogy strukturált lépéseken keresztül vezesse végig a felhasználót a profilalkotás során. Ez az oldal nem engedélyezi a továbbszaladást, amíg a kötelező adatok hiányoznak.

A megjelenítésért felelős belső UI elemek a DemographicFields és a NameFields komponensek. Ezek a modulok a frontend/src/components/profile/ könyvtárban helyezkednek el, és tartalmazzák az olyan apróbb, újrahasznosítható űrlap elemeket, mint a SelectField a nyelvek és vallások kiválasztásához, vagy a DateField a születési dátum megadásához. Ezek az elemek szorosan integrálódnak a React Hook Form könyvtárral, és a háttérben a Zod validációs motorját használják fel arra, hogy még a beküldés előtt, valós időben jelezzék, ha a felhasználó logikailag inkonzisztens adatokat próbálna rögzíteni. A profil módosítására a már bejelentkezett felhasználók esetében a profile komponens szolgál, amely oldalbetöltéskor aszinkron módon meghívja a REST API-t, és feltölti az űrlapot a meglévő adatokkal.

I.16. Párosítási és Keresési Modulok

A társkeresési élmény magját a partnerjelöltek megjelenítése és kezelése adja. A megjelenítés keretét a **MatchesLayout** komponens határozza meg, amely biztosítja, hogy a párosításokkal kapcsolatos nézetek egységes elrendezést és navigációs lehetőségeket kapjanak.

A tényleges adatok vizualizációja a **myMatches** oldal komponens feladata. Ez egy iterálható listát renderel a képernyőre, amely a felhasználó sikeresen visszaigazolt párosításait (Match objektumokat) jeleníti meg kártyás elrendezésben. Ha a felhasználó egy konkrét kártyára kattint, a rendszer a **matchProfile** oldalra navigálja, amely részletes betekintést nyújt a kiválasztott partner publikus demográfiai adataiba.

Ezeknek a komponenseknek az adatellátásáért a **matchesContext** felel. Ez egy globális React Context, amely egy központi memória tárolóként (store) működik. Amikor a kliens alkalmazás elindul, ez a kontextus lekéri az aktív párosításokat az API-tól, és eltárolja azokat. Így, amikor a felhasználó a profilok és a listák között navigál, a UI azonnal, várakozási idő nélkül képes frissülni, elkerülve a felesleges ismételt szerver hívásokat.

I.16. Valós Idejű Kommunikációs Komponensek

A platform technológiai csúcspontja az azonnali üzenetküldést és a valós idejű videós kapcsolatokat biztosító modul család. A rendszer két fő pillérre támaszkodik: a szövegalapú chatfelületre és a WebRTC alapú élő videó feedre.

Az asztali és mobil eszközökön egyaránt tökéletesen megjelenő chat felület elrendezést a **ChatColumn** komponens biztosítja. Ez a tartóelem fogja közre a beszélgetés teljes felületét. A tartalom dinamikus megjelenítését a **MessageList** komponens végzi. Ennek a komponensnek a feladata a chat előzmények intelligens kezelése. Belső állapottal (state) rendelkezik, és folyamatosan figyeli a felhasználó görgetési viselkedését. Amint a felhasználó a látható üzenetek tetejére görget, a komponens érzékeli ezt, és aszinkron hívást kezdeményez a backend felé a paginált üzenetek lekérésére. Az így visszakapott korábbi üzeneteket zökkenőmentesen fűzi hozzá a meglévő listához (végtelen görgetés), mindezt úgy, hogy a felhasználó fókusza ne ugorjon el az olvasott pozícióról. Az egyes üzeneteket a **MessageBubble** komponens formázza meg, vizuálisan élesen elkülönítve a saját és a partnertől kapott sorokat.

A hálózati réteg és a UI összekötéséért a **websocketContext** felel. Ez a kontextus a háttérben fenntart egy élő WebSocket kapcsolatot a szerverrel. Célja, hogy az alkalmazás bármely pontján elérhetővé tegye a valós idejű eseményeket. Ennek a kontextusnak a segítségével működik a **NotificationListener** komponens is, amely a bejövő eseményeket dolgozza fel. Például, amikor egy match megerősítésre kerül, egy toast komponens jelenik meg a képernyő alján. A **ChatInput** komponens szintén a

WebSocketContext-et használja fel a megírt üzenetek feladására.

I.17. Élő Videó Randi (WebRTC) és Állapotkezelés

Az alkalmazás legkomplexebb kliensoldali eleme a videós hívásokat menedzselő komponens, amely a WebRTC API-ra és a **WebSocketContext** által biztosított jelzőszerverre (signaling) épül. A vizuális felület és az üzleti logika vezérlését egy állapotgép (State Machine) végzi, a **ViewState** állapotváltozó segítségével. A felhasználói élmény az alábbi hat, jól elkülöníthető fázison halad keresztül:

1. **PERMISSIONS (Jogosultságok kérése):** A belépési pont. A komponens a böngésző natív API-jait használva engedélyt kér a felhasználótól a kamera, a mikrofon, valamint a geolokáció (GPS koordináták) használatára. Ezek az adatok elengedhetetlenek a helyi médiafolyam (**localStream**) létrehozásához és a földrajzi alapú párosításhoz.
2. **TESTING (Hardver tesztelése):** Miután a jogosultságok megadásra kerültek, a felhasználó egy előnézeti ablakban láthatja a saját kameraképét, és ellenőrizheti a beállításokat. A háttérben egy **AudioContext** és egy **AnalyserNode** fut, amely valós időben méri a mikrofon bemeneti jelszintjét (**micLevel**), vizuális visszajelzést adva arról, hogy a hang megfelelően működik-e.
3. **WAITING (Várakozás a partnerre):** Amikor a felhasználó elindítja a keresést, a rendszer a WebSocketen keresztül elküldi az aktuális GPS koordinátákat, majd belépteti a usert az aktív matchmaking poolba. A képernyőn egy pulzáló, keresést jelző animáció látható.
4. **CONNECTING (Kapcsolat felépítése):** Amint a háttérben futó algoritmus megtalálja a megfelelő partnert, a komponens megkapja a partner profil adatait, és megkezdődik az aszinkron WebRTC jelzési (signaling) folyamat. Létrejön az **RTCPeerConnection** példány. A kliensek Google STUN és privát TURN szerverek segítségével megosztják egymással a hálózati adatokat (ICE candidates), valamint az SDP ajánlatokat és válaszokat (Offer/Answer).
5. **IN_CALL (Aktív Hívás):** A közvetlen peer-to-peer kapcsolat sikeresen felépül. A felületen megjelenik a partner élő videóképe (**remoteStream**), a saját kép pedig egy "kép a képben" (PiP) ablakba kerül. A dinamikus élmény fenntartása érdekében a rendszer elindít egy 120 másodperces visszaszámlálót. Ha az idő lejár, vagy bármelyik fél a megszakítás gombra kattint, a kapcsolat azonnal bontásra kerül.
6. **FEEDBACK (Értékelési fázis):** A hívás befejeztével a rendszer automatikusan lekéri a partner részletes profil adatait, majd a felhasználónak értékelnie kell a

találkozást.

7. **Hívás utáni interakciók és Moderáció:** A Feedback (értékelés) nézeten a rendszer kötelező interakciót vár a felhasználótól, mielőtt újraindíthatná a keresést:
 - **Like és Pass:** A felhasználó a "Thumbs Up" (Like) vagy "Thumbs Down" (Pass) gombokkal minősítheti a beszélgetést. Ez az aszinkron visszacsatolás a backend felé kerül továbbításra. Amennyiben mind a ketten a like gombra kattintottak, az adatbázisban létrejön a match.
 - **Jelentés és Tiltás (Report and Block):** A közösségi irányelvek betartatása érdekében a felületen helyet kapott egy Flag (Zászló) ikon, amely megnyitja a jelentési modált. Amennyiben a partner nem megfelelő viselkedést tanúsított, a felhasználó megadhatja az okot és beküldheti a jelentést. Ez a művelet nem csak az adminisztrátorokat értesíti, hanem azonnal és véglegesen blokkolja a két felhasználó közötti jövőbeli interakciókat.

I.18. Manuális tesztek

A szoftver kiemelkedő minőségének, adatbiztonságának és a zökkenőmentes felhasználói élménynek a garantálása érdekében rendkívül átfogó manuális tesztelési stratégiát hajtottunk végre. A manuális tesztelés elengedhetetlen a felhasználói felület ergonómiájának, a választások érzékelésének és a böngésző-specifikus anomáliáknak a feltárására. A tesztelési metodika tudatosan szétválasztja az üzemszerű működést szimuláló (pozitív) tesztutakat, és a szándékosan hibás bemenetekkel, illetve jogosulatlan hozzáférésekkel provokált (negatív) tesztutakat.

A tesztelés egyik legfontosabb sarokköve a **biztonság és az útvonalvédelem verifikációja** volt. Ezen a ponton az ellenőrzések arra fókuszáltak, hogy a React Router által védett felületeket (mint például a beállítások vagy a chat oldalak) kikerülheti-e egy nem hitelesített felhasználó az URL közvetlen manipulációjával. A tesztek igazolták, hogy a hitelesítést ellenőrző mechanizmusok azonnal, még az oldal tartalmának kliensoldali renderelése előtt közbelépnek, és a látogatót visszairányítják a biztonságos bejelentkezési felületre. Hasonlóan szigorú ellenőrzésen esett át a **munkamenet-logika és a visszaút-védelem**. Ha egy már aktív munkamenettel rendelkező, bejelentkezett felhasználó megpróbálja elérni a /login útvonalat, vagy a böngésző "Vissza" gombjával navigálna oda, a rendszer intelligensen felismeri a

meglévő JWT tokenet, és ahelyett, hogy a bejelentkezési formot jelenítené meg, azonnal átirányítja őt az alkalmazás főoldalára.

A felhasználói élmény szempontjából kritikus fontosságú volt a vizuális visszajelzések tesztelése. Az **aszinkron Toast-alapú visszajelzések** működését minden olyan műveletnél vizsgáltuk, amely adatbázis módosítással jár. Legyen szó a preferenciák sikeres frissítéséről, vagy a jelszó-visszaállítási kérelem beküldéséről, a rendszernek egyértelmű, meghatározott idő után automatikusan eltűnő üzenettel kell jeleznie a művelet kimenetelét a felhasználónak. Emellett a hálózati kommunikáció alatti **betöltési állapotok (loading states)** kezelése is tesztelésre került. A tesztek során meggyőződünk arról, hogy az API hívások időtartama alatt a beküldő gombok inaktívvá válnak, és egy betöltést jelző animáció jelenik meg. Ezzel a mechanizmussal sikeresen kiküszöböltük a "dupla kattintásból" eredő adat szemetelést és a szerver felesleges terhelését.

A manuális tesztek eredményeit és lépéseit az alábbi, részletesen kidolgozott táblázatok foglalják össze.

Részletes tesztelési táblázat – Felhasználói felület (Frontend)

Tesztelt Folyamat	Teszt Lépéseinek Részletes Leírása	Várt Rendszerválasz és Eredmény
Teljes regisztrációs és belépési életciklus	1. Navigáció a /register oldalra. 2. Űrlap kitöltése valós, nem létező e-mail címmel. 3. Beküldés. 4. Átirányítás után bejelentkezés az új adatokkal.	A regisztráció után a rendszer zöld Toast üzenetet küld. A bejelentkezést követően az alkalmazás a profil beállítása oldalra irányít. Nincs hiba a konzolon.
Hibás regisztráció (Negatív út)	1. Navigáció a /register oldalra. 2. Jelszó mezőbe "Titkos123" beírása. 3. Megerősítő jelszóba "Titkos321" beírása. 4. Regisztráció gomb megnyomása.	A kliensoldali Zod validátor másodpercek töredéke alatt piros hibaüzenetet jelenít meg a mező alatt. A kérés hálózati szinten el sem indul a szerver felé.
Preferencia logikai validáció	1. Bejelentkezett állapotban navigáció a profil oldalra. 2. A minimum elvárt életkor mező módosítása 30-ra. 3. A maximum életkor mező módosítása 25-re. 4. Mentés gomb kattintása.	Az űrlap beküldésre kerül, de a backend API 400 Bad Request hibával tér vissza. A frontend a hibaüzenetet értelmezi, és piros Toast üzenetben értesíti a felhasználót az érvénytelen tartományról.

Csevegés üzeneteinek lapozása	1. Meglévő párosítás megnyitása a /my-matches felületről. 2. Görgetés az üzenetlista legtetejére. 3. Várakozás a kérés lefutására.	A rendszer észleli a lista végét, meghívja az API-t. Az új üzenetek megérkeznek, a görgetősáv pozíciója megmarad az olvasott üzenetnél, nem ugrik az oldal tetejére.
Valós idejű WebSocket kommunikáció	1. Két különböző inkognitó böngésző megnyitása. 2. Belépés "A" és "B" fiókkal, akik párosítva vannak. 3. "A" felhasználó üzenetet küld "B"-nek a csevegőablakban.	A küldés gomb megnyomása után az üzenet azonnal, oldalfrissítés és látható késleltetés nélkül megjelenik "B" felhasználó képernyőjén a megfelelő stílusozással.
Ergonómia és mobil reszponzivitás	1. Az alkalmazás megnyitása asztali böngészőben. 2. A böngészőablak fokozatos méretezése mobil szélességig (viewport váltás).	A navigációs sáv zökkenőmentesen hamburger menüvé alakul. A több oszlopos elrendezések egymás alá rendeződnek, az elemek nem csúsznak ki a képernyőről, a betűméretek olvashatóak maradnak.

Részletes tesztelési táblázat – Adminisztrátori felület

A rendszer biztonsági architektúrája miatt az adminisztrátori jogosultságok ellenőrzése kiemelt fontosságú volt a tesztelési fázisban.

Tesztelt Folyamat	Teszt Lépéseinek Részletes Leírása	Várt Rendszerválasz és Eredmény
Jogosulatlan admin hozzáférési kísérlet	1. Belépés hagyományos "Felhasználó" (User) jogosultságú fiókkal. 2. Böngésző címsorába a /admin/dashboard URL manuális beírása és megnyitása.	A frontend biztonsági komponense (Guard) érzékeli, hogy a memóriában lévő JWT token nem tartalmaz admin szerepkört. A felület nem renderelődik ki, a rendszer hibaüzenet kíséretében visszairányítja a felhasználót a kezdőoldalra.
Backend API végpont védelme	1. API tesztelő eszköz (pl. Postman) megnyitása. 2. Normál felhasználó JWT tokenjének beállítása a fejlécben. 3. Lekérdezés indítása egy kizárólag adminok számára elérhető statisztikai végpont felé.	A FastAPI jogosultságkezelő middleware-je dekódolja a tokent, felismeri a jogosultság hiányát, és a belső üzleti logika végrehajtása előtt HTTP 403 Forbidden státuszkóddal elutasítja a kérést.

I.19. Automatizált tesztek

A manuális tesztek egy rendkívül szigorú és kiterjedt automatizált tesztelési környezet egészíti ki. Az automatizált tesztek biztosítják az úgynevezett regresszió mentességet, azaz garantálják, hogy a jövőbeli fejlesztések vagy kód-refaktorálások nem teszik tönkre a már stabilan működő funkciókat. Az alkalmazás két teljesen különálló automatizált tesztelési réteggel rendelkezik a backend és a frontend kódok verifikálására.

A backend API tesztelése a Python ökoszisztémában standardnak számító Pytest keretrendszerrel történik. Ezek a tesztek szintén két fő logikai csoportra oszthatók. Az integrációs tesztek (amelyek a tests/integration/ könyvtárban találhatóak) az API útvonalak, az üzleti logika és az adatbázis valós interakcióit vizsgálják egy memóriában futó tesztadatbázis segítségével. Például a test_auth.py szkript automatikusan végigfuttat több száz belépési kísérletet helyes és szándékosan elrontott adatokkal, ellenőrizve a kapott JWT tokenek érvényességét. A test_matches.py integrációs teszt két felhasználó párosítását, megerősítve azt, hogy az adatbázis egyediségi megkötései (UniqueConstraint) a gyakorlatban is megakadályozzák az ismétlődő kapcsolatok létrejöttét. Ezzel szemben a backend egységtesztek (unit tesztek, a tests/unit/ mappában) az adatbázis teljes kizárásával, izoláltan vizsgálják a matematikai és logikai függvényeket. A test_background_matchmaking_unit.py úgynevezett "mockolt" (hamisított) bemeneti adatokkal ellenőrzi, hogy a párosító algoritmus képes-e ezredmásodpercek alatt helyes döntést hozni, és hogy garantáltan soha nem hoz-e össze olyan felhasználókat, akiknek a beállított preferenciái ellentmondanak egymásnak.

A frontend és a felhasználói folyamatok végponttól végpontig (End-to-End, E2E) történő automatizált tesztelésére egy teljesen különálló C# alapú projekt (VidateTests/) szolgál. Ez a tesztkörnyezet Selenium alapokon nyugszik, és a modern tesztelési elveknek megfelelően a Page Object Model (POM) tervezési mintát alkalmazza. Ennek lényege, hogy a tesztek logikai egységekre bontják a weblapot; a DOM elemek (mint gombok, beviteli mezők, felugró ablakok) azonosítói rejtve maradnak olyan osztályokban, mint a LoginPage.cs vagy a ProfileSetupPage.cs. Ha a weboldal dizájnya megváltozik, elegendő csak ezeket az osztályokat frissíteni, a Tests/ mappában található funkcionális teszt forgatókönyveket nem kell átírni. A SetupProfileTests.cs szkript például képes elindítani egy valós, automatizált böngészőt, amely egy másodpet töredéke alatt kitölt egy teljes felhasználói regisztrációs űrlapot, kiválasztva a több

opciós legördülő menük elemeit is, majd leellenőrzi, hogy a képernyőn megjelent-e a sikeres mentést igazoló visszaigazolás. A MyMatchesTests.cs pedig a komplex DOM elemek – mint a dinamikusan betöltődő csevegő ablakok – interakcióit vizsgálja, kattintásokat szimulálva az üzenetküldés gombokon.

II. Felhasználói dokumentáció

A felhasználói dokumentáció célja, hogy az alkalmazást használó különböző szereplők számára világos, gyakorlatias és átfogó képet nyújtson a platform rendeltetéséről. Ez a fejezet bemutatja azt a piaci és társadalmi problémát, amelyet az alkalmazás megoldani hivatott, részletezi az elkülönülő jogosultsági szinteket, és konkrét felhasználói eseteken (Use Cases) keresztül szemlélteti a szoftver üzleti logikáját.

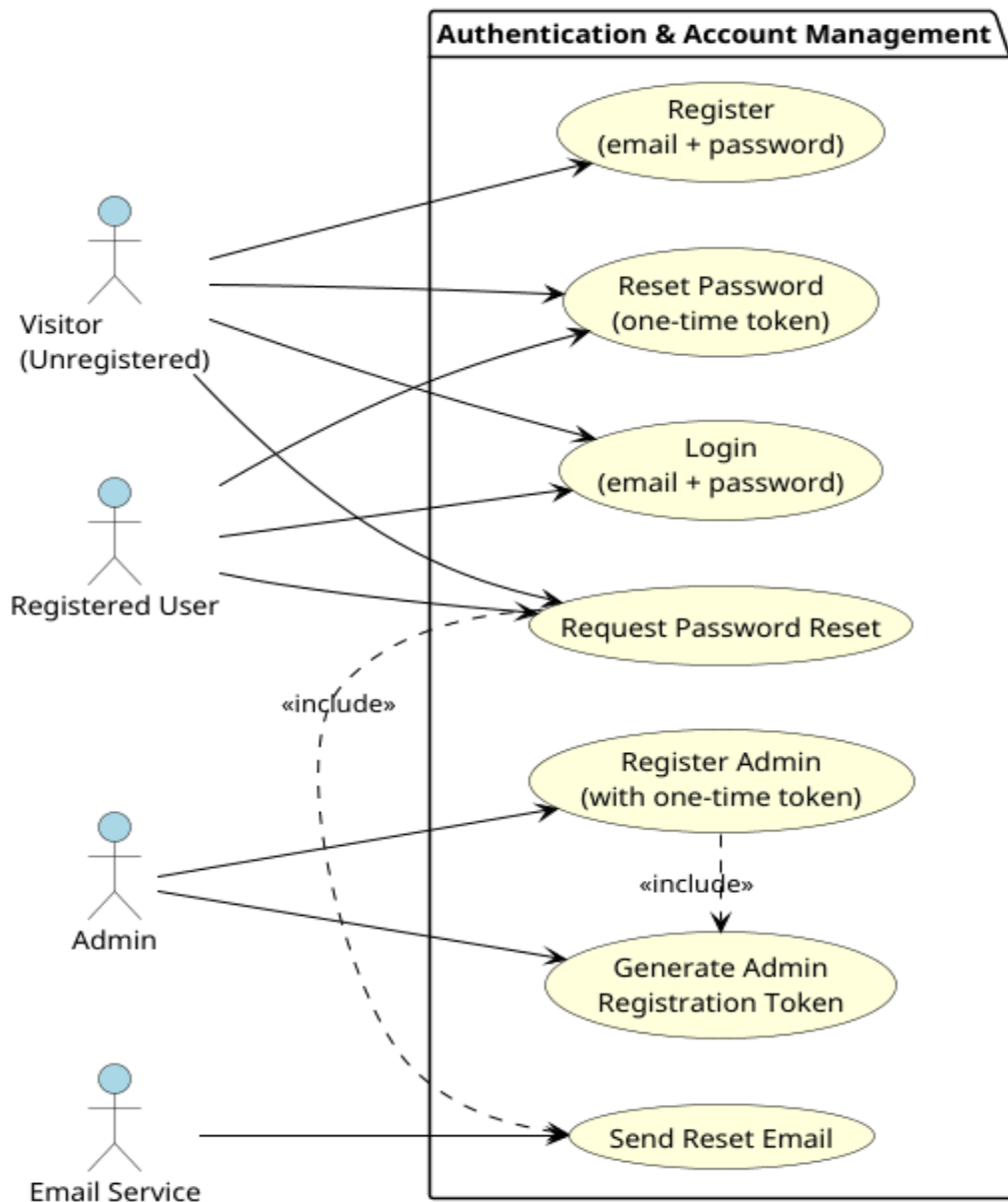
II.1. A probléma leírása

A modern digitális térben, különösen az online társskereső piacon, a felhasználók folyamatosan két alapvető problémával szembesülnek: az irreleváns, felületes párosítások frusztrálóan magas számával, valamint a kapcsolattartást nehezítő technológiai és vizuális korlátokkal. A hagyományos, tömegpiaci társskereső alkalmazások túlnyomó többsége a felszínes jellemzőkre fókuszál, miközben a kommunikáció kimerül az aszinkron szöveges üzenetküldésben. Ez a megközelítés magas felhasználói elégedetlenséghez, hamis profilok (catfishing) elszaporodásához és a platformok gyors elhagyásához vezet.

A másik jelentős probléma technológiai gyökerű. A párosított felhasználók közötti kapcsolatfelvétel gyakran lassú, a személyes találkozó megszervezése pedig hetekig is elhúzódhat.

A Vidate projekt elsődleges célja egy olyan platform létrehozása, amely radikális választ ad mindkét kihívásra. A szoftver egy komplex preferencia rendszeren alapuló (életkor, gyermekvállalási szándék, dohányzási szokások, preferált beszélt nyelvek, vallás, földrajzi távolság) alkalmazás segítségével intelligensen szűri és pontozza a potenciális partnereket. Továbbá az interakciót a modern WebRTC technológia integrálásával emeli új szintre, amely garantálja a valós idejű, azonnali élő videós kapcsolatfelvételt a felek között. Ezáltal a szoftver azonnali, személyes és dinamikus élményt nyújt, minimalizálva a frusztrációt, és a tartós, értékes emberi kapcsolatok kialakítására helyezve a hangsúlyt.

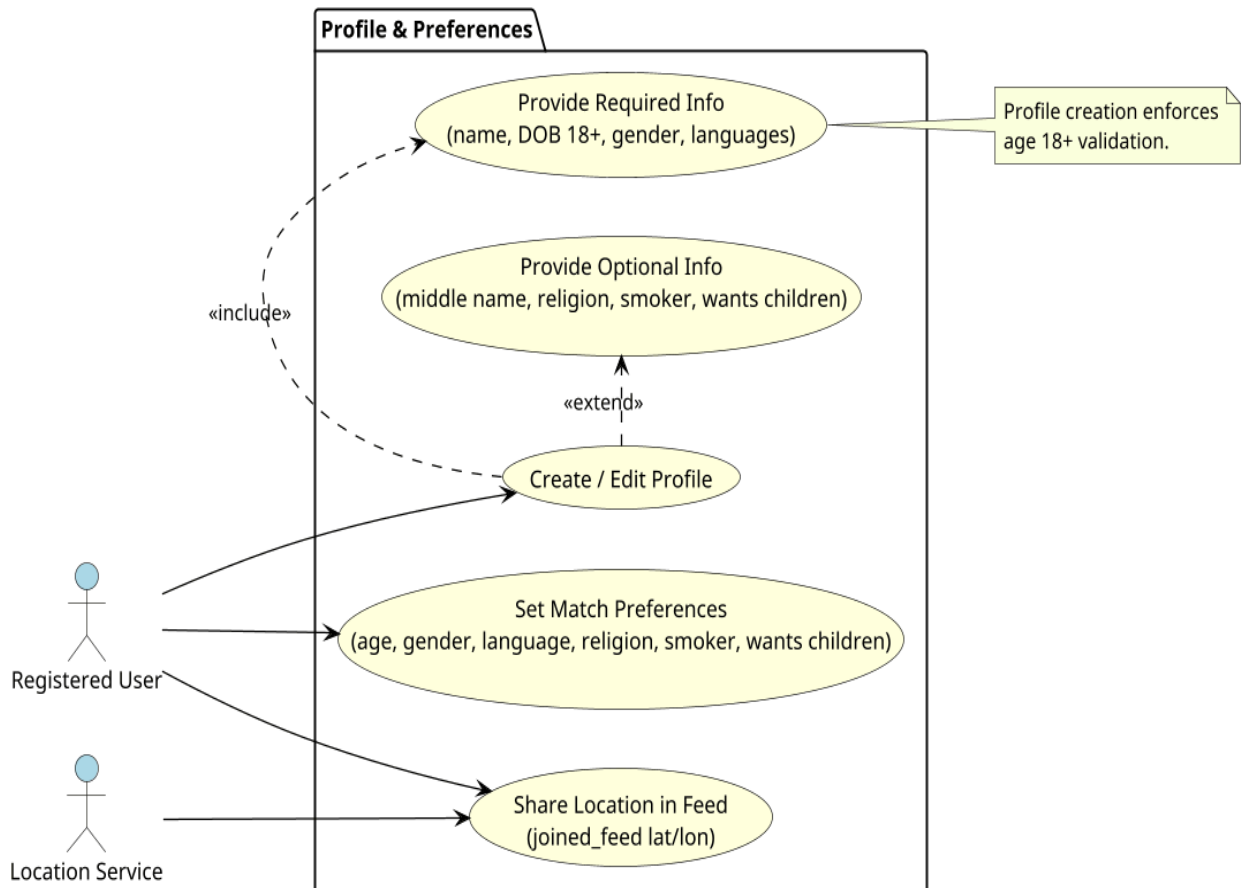
II.2. Szerepkörök bemutatása



A Vidate alkalmazás egy szigorúan ellenőrzött, többszintű jogosultsági modellt alkalmaz a rendszer stabilitásának, a felhasználók személyes adatainak és a platform integritásának védelme érdekében. A rendszer alapvetően három különböző szerepkört különböztet meg:

1. **A Látogató (Nem regisztrált felhasználó):** A legalsó szintű szerepkör. Bárki, aki felkeresi az alkalmazás nyilvános webcímét, ebbe a kategóriába esik. A látogató hozzáférési köre rendkívül limitált: kizárólag a platform nyilvános bemutatkozó

felületét böngészheti, ahol megtekintheti a szolgáltatás előnyeit bemutató marketing anyagokat. Az adatvédelmi irányelvek értelmében a rendszer belső funkcióihoz, a videó feedhez és mások profiljaihoz egyáltalán nem férhet hozzá. A látogató által végrehajtható egyetlen proaktív cselekvés a belépési, a regisztrációs, vagy a jelszó-visszaállítási űrlapok elindítása.



2. **A Felhasználó (Regisztrált tag):** A platform aktív, hitelesített szereplője. A sikeres regisztráció és a profil beállítása után a felhasználó a rendszer teljes jogú résztvevőjévé válik. Létrehozhatja és módosíthatja a saját demográfiai profilját és társkeresési preferenciáit. Beléphet a valós idejű "Matchmaking poolba", ahol az algoritmus partnereket keres számára, majd a WebRTC videó feed segítségével élő videó randikon vehet részt. A magánszféra védelme érdekében a felhasználó kizárólag a saját fiók adatait módosíthatja, és a platform teljes felhasználói adatbázisában történő szabad böngészés természetesen nem engedélyezett.
3. **Az Adminisztrátor (Admin):** A rendszer legmagasabb szintű, kiváltságos jogosultsággal rendelkező szerepköre. Az adminisztrátorok elsődleges feladata

a rendszer globális felügyelete és a moderációs irányelvek betartatása. Számukra egy teljesen különálló, rejtett admin dashboard érhető el. Ezen a felületen az adminisztrátorok jogosultak a felhasználók által jelentett (Reported) visszaélések kivizsgálására, valamint a szabályszegő fiókok végleges felfüggesztésére. Ugyancsak az ő hatáskörükbe tartozik a rendszer statikus szótártábláinak (új nyelvi vagy vallási kategóriák felvitele) kezelése. Illetve a a dashboard-on belül lehetőség van új admin fiókok létrehozására is. Minden adminisztrátori műveletet JWT alapú ellenőrzés véd.

II.3. Felhasználói esetek (Use Cases) és Üzleti Logika

A következőkben az alkalmazás legkritikusabb üzleti folyamatait és felhasználói útjait mutatjuk be, rávilágítva arra, hogy a képernyőn látható egyszerű lépések mögött milyen komplex algoritmusok dolgoznak.

Profil beállítása és preferencia logikák

Amikor egy frissen regisztrált látogató először jelentkezik be az alkalmazásba, a rendszer azonnal a Profil Beállítása (setupProfile) oldalra irányítja. A szoftver algoritmusa nem képes működni profil adatok hiányában, így az oldal elhagyása a kitöltés befejezése előtt blokkolva van.

- A folyamat első fázisában a felhasználónak nyilatkoznia kell a saját valós demográfiai adatairól (nem, születési idő, beszélt nyelvek, vallás, dohányzási szokások).
- A második fázisban az elvárásokra, a "Preferenciákra" terelődik a fókusz. A felhasználó egy újabb űrlapon határozza meg a jövőbeli partnerre vonatkozó kritériumokat. Ha a felhasználó tévedésből ellentmondásos adatokat ad meg (pl. a maximum életkor kisebb, mint a minimum), a frontend azonnali vizuális figyelmeztetéssel blokkolja a mentést.
- Az űrlap beküldésekor az adatok elindulnak a szerver felé, bekerülnek a PostgreSQL adatbázisba, és a profil aktívvá válik.

Keresés indítása és a matchmaking algoritmus

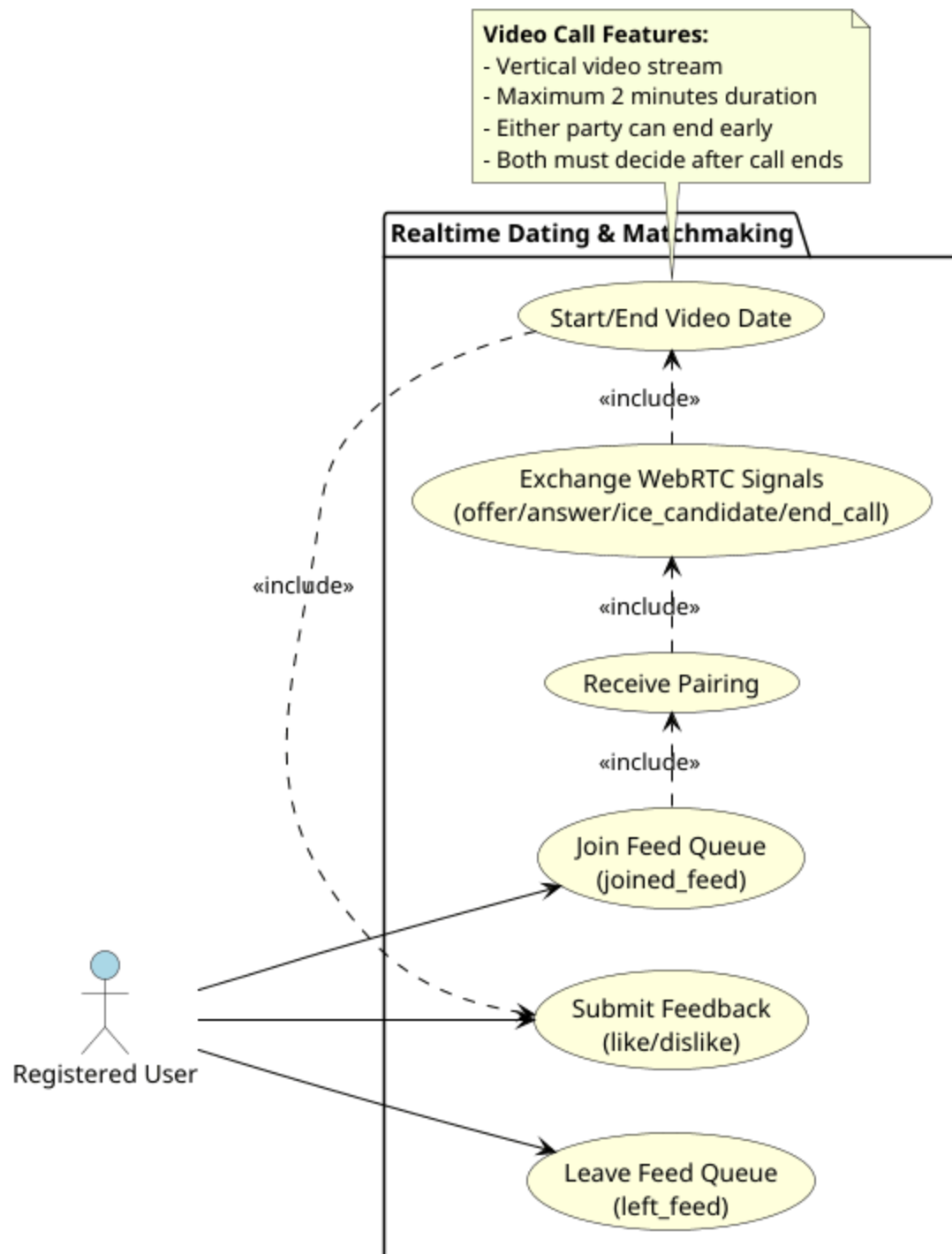
Amikor a felhasználó a videó feed felületen elindítja a keresést, az alkalmazás natív böngésző API-kat használva bekéri a geolokációs (GPS) koordinátáit, amelyeket a Redis a memóriába tölt a személyre szabottabb élmény érdekében

A hagyományos, "tökéletes egyezést" váró rendszerekkel ellentétben a Vidate a

hátterben egy hibrid, többtényezős pontozó algoritmust futtat aszinkron módon:

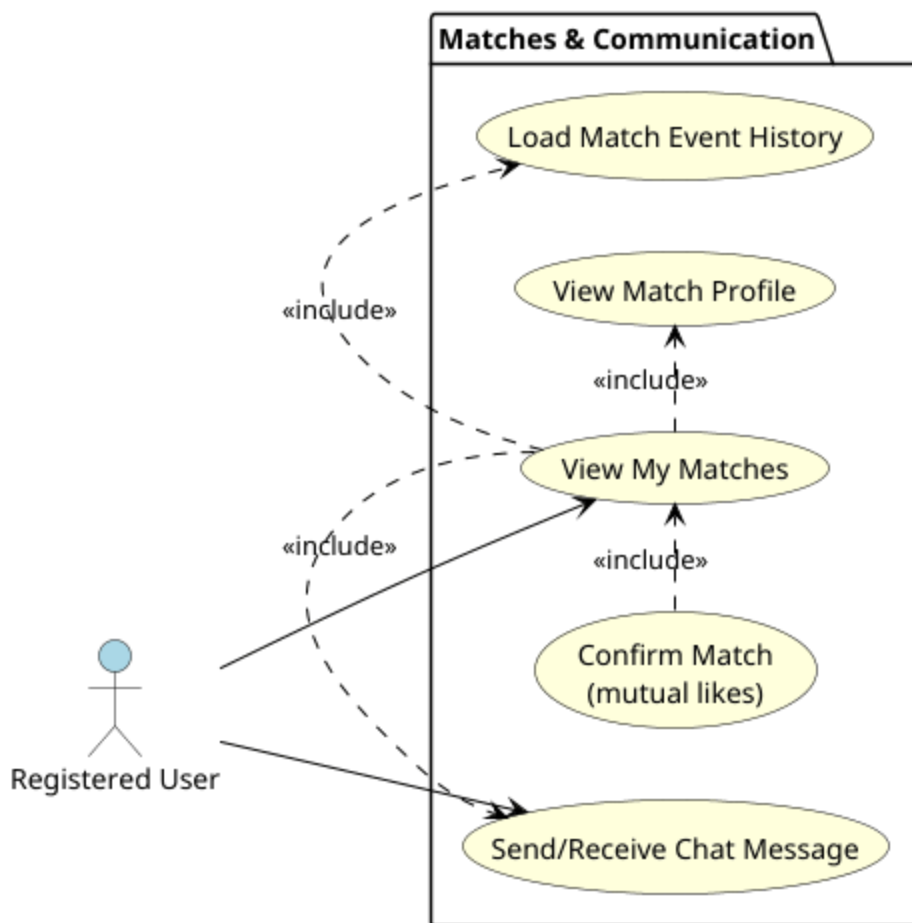
- **Hard Constraints (Kizáró okok):** A rendszer azonnal kiszűri a nem megfelelő nemű, illetve a moderációs rendszerben korábban már letiltott (blocked) partnereket.
- **Soft Constraints (Puha korlátok):** A megmaradt jelölteket az algoritmus súlyozva pontozza. Hatalmas büntetőpontot kapnak azok, akikkel a felhasználó nemrég már beszélt (elkerülve az ismétlődő találatokat). Bónuszpont jár a közös nyelvért, a megfelelő korosztályért és a fizikai közelségért. Amint a Lua script atomi tranzakcióval kiválasztja a legmagasabb pontszámú szabad partnert, a rendszer WebSocketen keresztül azonnali jelzést küld mindkét félnek a sikeres találatról.

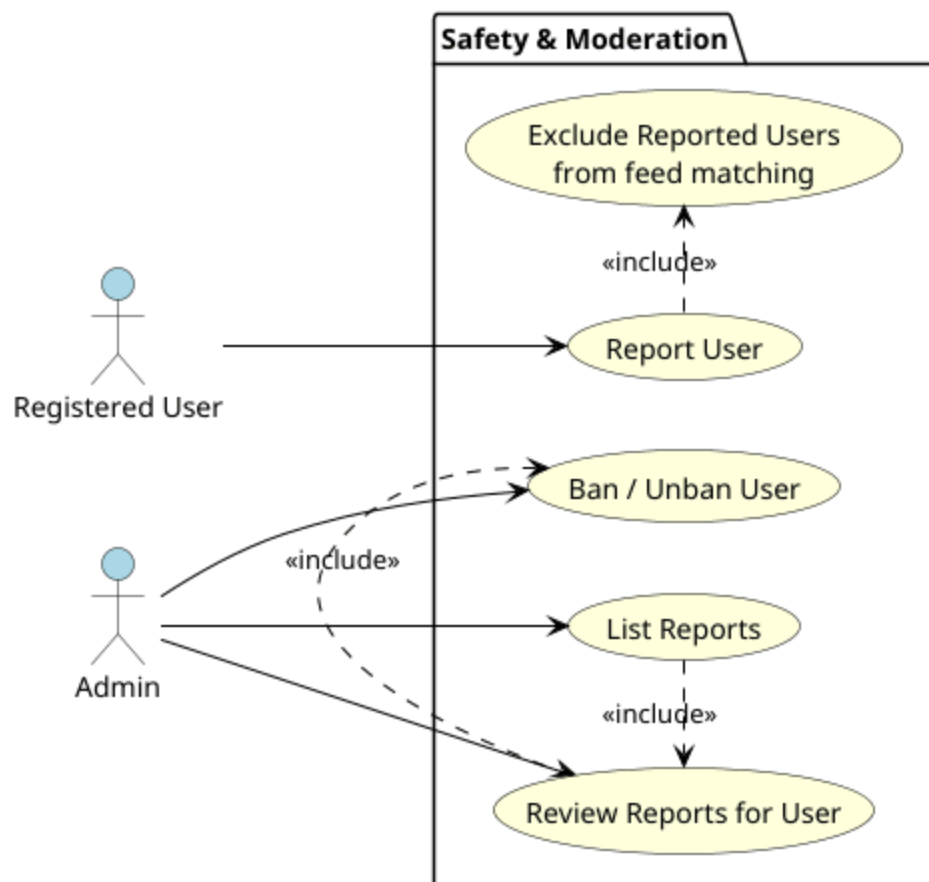
Élő Videórandi (WebRTC) lebonyolítása



A platform technológiai csúcspontja a videó feed, amely a sikeres párosítás után azonnal életbe lép. A folyamat egy szigorú állapotgépen (State Machine) halad keresztül:

- **PERMISSIONS és TESTING:** A rendszer engedélyt kér a kamera és mikrofon használatára. A felhasználó egy előnézeti ablakban ellenőrizheti a képét és a beépített hangerő mérő segítségével a mikrofonját.
- **CONNECTING (Kapcsolódás):** A WebSocket jelzőszerveren (signaling server) keresztül a két böngésző a háttérben azonnal kicseréli a hálózati útválasztási adatokat (SDP és ICE candidate-ek), majd a STUN/TURN szerverek segítségével áthatolnak a NAT-tűzfalakon.
- **IN_CALL (Aktív Hívás):** Közvetlen, peer-to-peer kapcsolat épül fel. A képernyőn megjelenik a partner élő videóképe. A beszélgetés dinamikájának fenntartása érdekében elindul egy 120 másodperces visszaszámláló. Bármelyik fél gombnyomásra, vagy az idő lejártával megszakíthatja a hívást.
- **FEEDBACK (Értékelés):** A hívás után a felek azonnal visszacsatolást adnak. A "Like" (Tetszik) vagy "Pass" (Tovább) gombok aszinkron módon frissítik az adatbázist, meghatározva a jövőbeli találatok esélyét. Ha mindkét fél "Like"-ot nyomott, a kapcsolat a "Kapcsolataim" (Matches) listába kerül, ahol a jövőben szöveges üzeneteket válthatnak a Chat felületen.





Moderáció és Biztonság

Egy élő videós randialkalmazás esetében a felhasználók biztonsága kritikus. A feedback fázisban elérhető a "Jelentés és Tiltás" (Report and Block) funkció egy zászló ikon formájában. Ha a partner nem megfelelő viselkedést tanúsít, a felhasználó megszakíthatja a kapcsolatot, és a megadott okkal jelentheti a fiókot. Ez a művelet:

1. Véglegesen blokkolja a két felhasználó közötti jövőbeli párosítást (Hard constraint).
2. Bejegyzést hoz létre a PostgreSQL adatbázisban, amelyet az adminisztrátorok a saját felületükön felülvizsgálhatnak a fiók esetleges végleges letiltásához.

Biztonságos Fiók-helyreállítás és Jelszókezelés

A bejelentkezési nehézségekkel küzdő felhasználó az "Forgot password" opcióra kattintva elindíthatja a helyreállítási protokollt. A rendszer adathalászat elleni védelemből nem igazolja vissza a képernyőn az e-mail cím létezését. A backend egy egyszer használatos, 15 percig érvényes kriptográfiai tokent generál, amelyet e-mailben küld el. A hivatkozásra kattintva a felhasználó egy új jelszót adhat meg. A rendszer az új jelszóból friss Argon2id lenyomatot képez.

III. Idézett munkák

1. FastAPI (<https://fastapi.tiangolo.com/>)
2. SQLAlchemy (<https://sqlmodel.tiangolo.com/tutorial/fastapi/>)
3. SQLAlchemy ORM (<https://docs.sqlalchemy.org/en/20/>)
4. Pydantic (<https://docs.pydantic.dev/>)
5. Uvicorn (<https://www.uvicorn.org/>)
6. pytest (<https://docs.pytest.org/en/stable/>)
7. pytest-asyncio (<https://pytest-asyncio.readthedocs.io/en/latest/>)
8. Redis (<https://redis.io/docs/latest/>)
9. Redis Stack – Redis Stack szerver
(https://redis.io/docs/latest/operate/oss_and_stack/stack/)
10. PostgreSQL adatbázis (<https://www.postgresql.org/docs/>)
11. Docker (<https://docs.docker.com/>)
12. Docker Compose (<https://docs.docker.com/compose/>)
13. Nginx (<https://nginx.org/en/docs/>)
14. Cloudflare Tunnel
(<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>)
15. React (<https://react.dev/>)
16. React Router (<https://reactrouter.com/>)
17. React Hook Form (<https://react-hook-form.com/get-started>)
18. Axios (<https://axios-http.com/docs/intro>)
19. Vite (<https://vite.dev/guide/>)
20. Tailwind CSS (<https://tailwindcss.com/docs>)
21. Flowbite React (<https://flowbite-react.com/>)
22. Selenium WebDriver (<https://www.selenium.dev/documentation/webdriver/>)
23. NUnit (<https://docs.nunit.org/>)